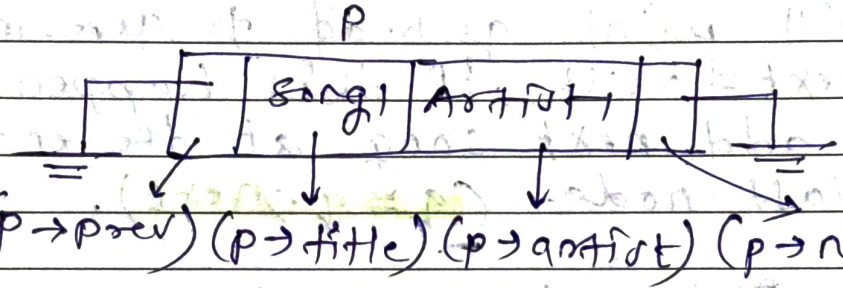
* case study (DSA)

Page No. 1
Date

* Music playlist Manager using concept of Doubly linked list. (Menu driven).

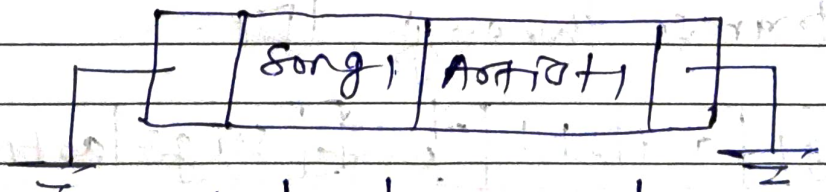
→ ① Initial play list (Show head, current, prev and next pointer).

Let "p" be the pointer having address of newly allocated song.



where "p→prev" is address of previous song.
"p→title" is title of song stored in "p".
"p→artist" is name of artist stored in "p".
"p→next" is address of next song.

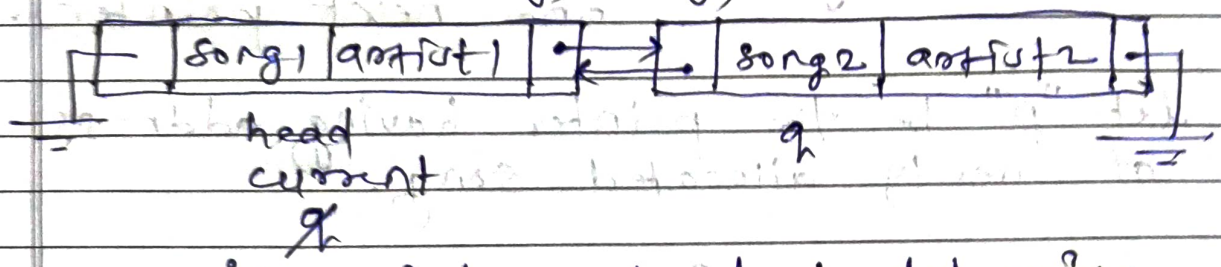
⇒ If this song (node) is 1st song of play list then
head = p and current = head;



p, head, current.

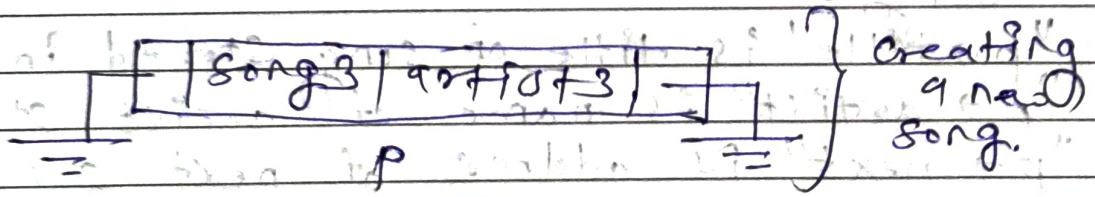
② Insertion - show linked list structure before & after adding a new song.

let before adding song playlist was like,



we will point q=head to determine q->next = NULL, if this happens, we will add next song at the ending of last node (q = q->next).

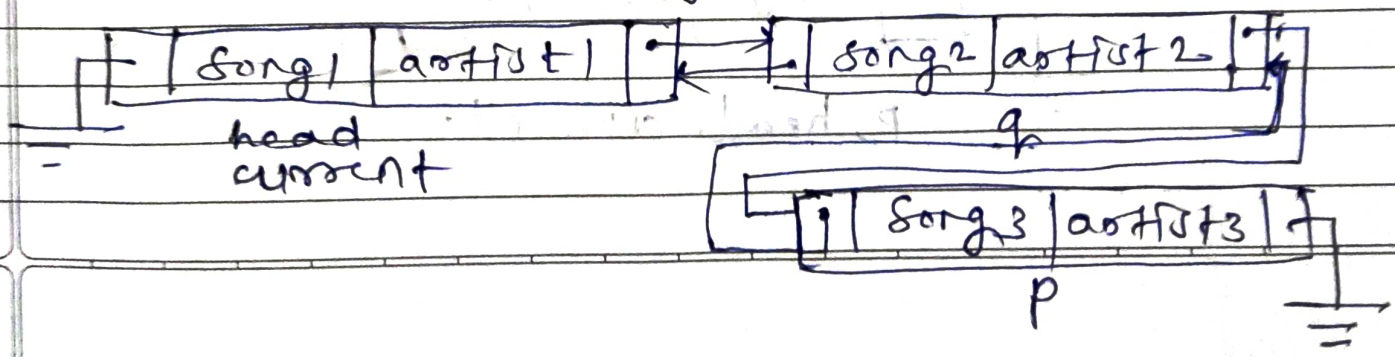
let a statement is called,
Song *p = new Song (x, y);



x = title and y = artist.

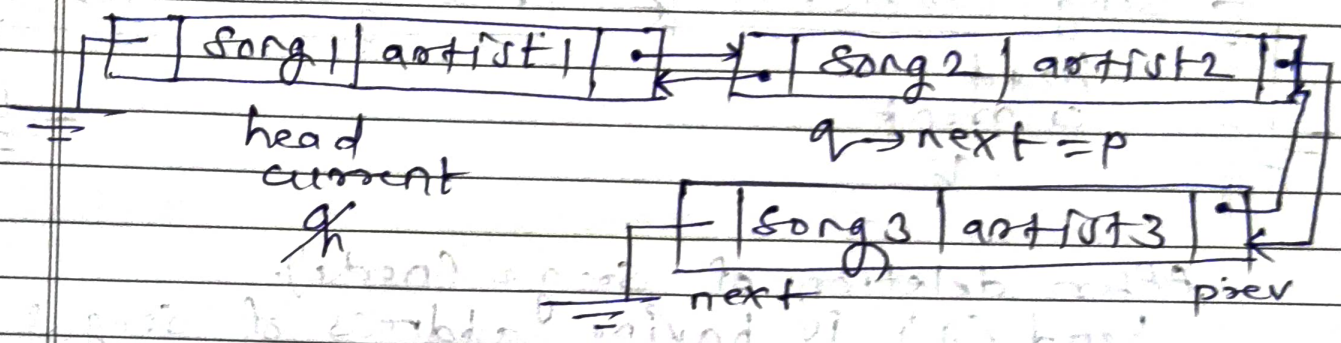
if head ≠ NULL (head != NULL)
p->prev = q; } giving address
q->next = p; } to each other.

after
q will traverse through list.



* (8) deletion : (show linked list structure before & after deleting a song and reconnection of remaining song).

let playlist is,



let we have to delete song2 = x

head q = head (pointing q to head).

check q->title == x;

if yes.

head = head->next;

current = current->next;

delete q;

return;

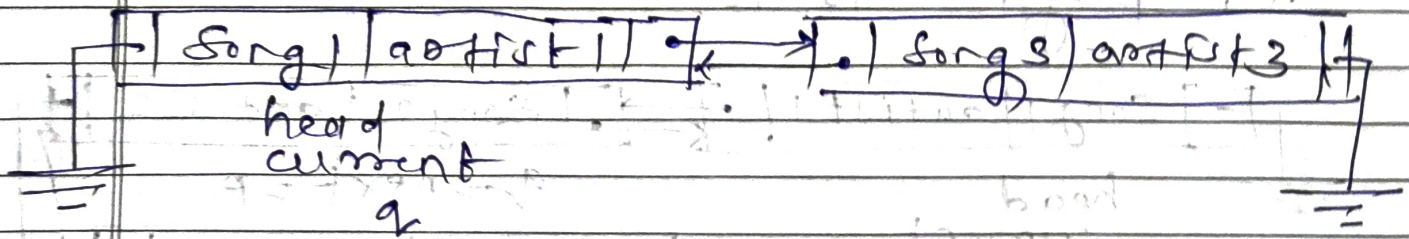
~~if not => q=q->next;~~ { initially } { p=NULL }.

~~check => q->title==x~~

~~{if yes point another pointer p}~~

```
while (q->next != NULL)
do if (q->next->title==x)
     p = q->next;
     q->next = p->next;
     p->next->prev = q;
     delete p;
```

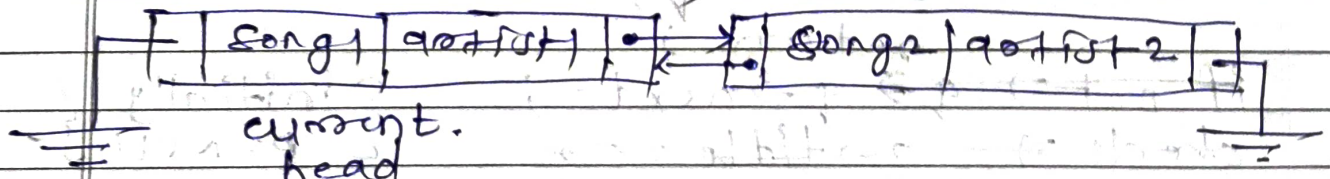
So "p" will be deleted and q→next will be having address of song3 and song1 (node) will be having address of p→next.



After deletion of song2 (node).
head (q) is having address of song3 node and song3 node is having address of song1 node.

4. Next Navigation. (Show how current moves from one song to the next).
(current = current→next)
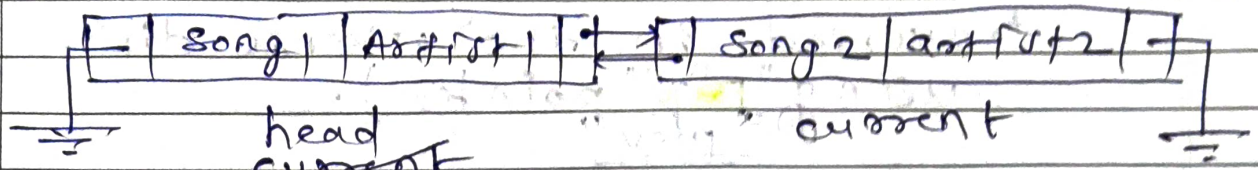
(before)



a function called as play

```
void play (string x)
if (x == "next")
{ if (current->next == NULL)
    cout << "last song, can't play next";
  else
    current = current -> next;
}
  cout << "current song: " << current->title;
```

after)

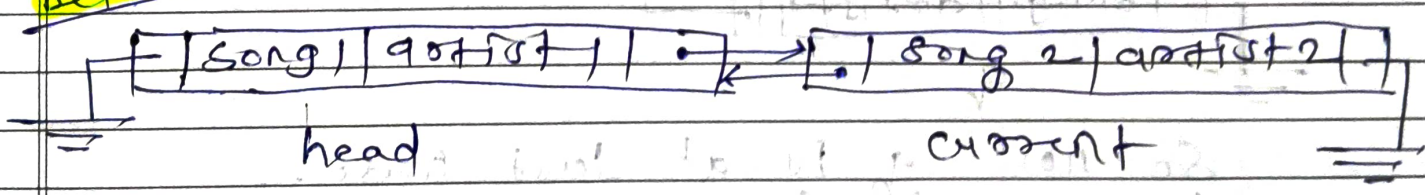


at output
current song : song 2
will be displayed

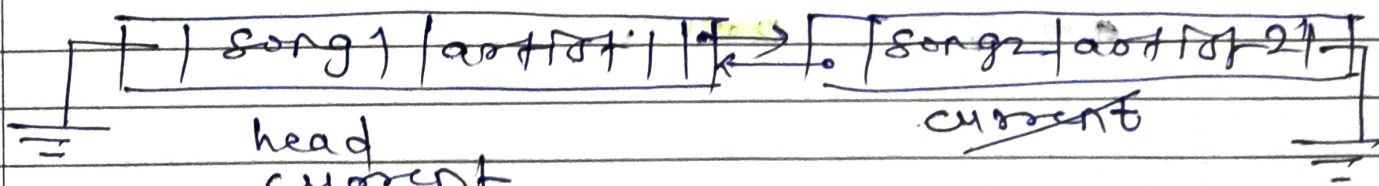
* ⑤ Previous Navigation : Show how current moves from one song to other using (current = current → prev.)

```
else if (x == "previous")
  { if (current → previous == NULL)
      cout << " first song cannot play previous song " ;
    else
      current = current → prev;
  }
```

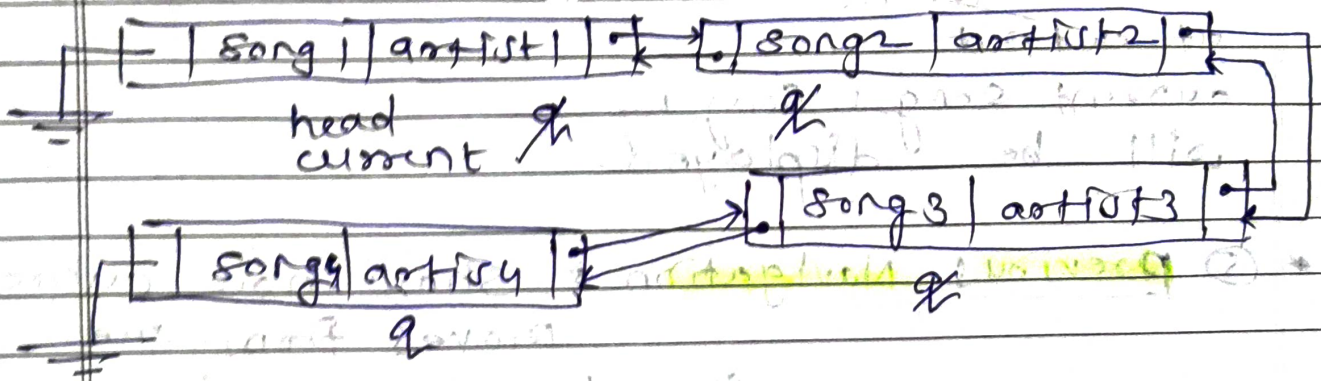
before



after calling function.

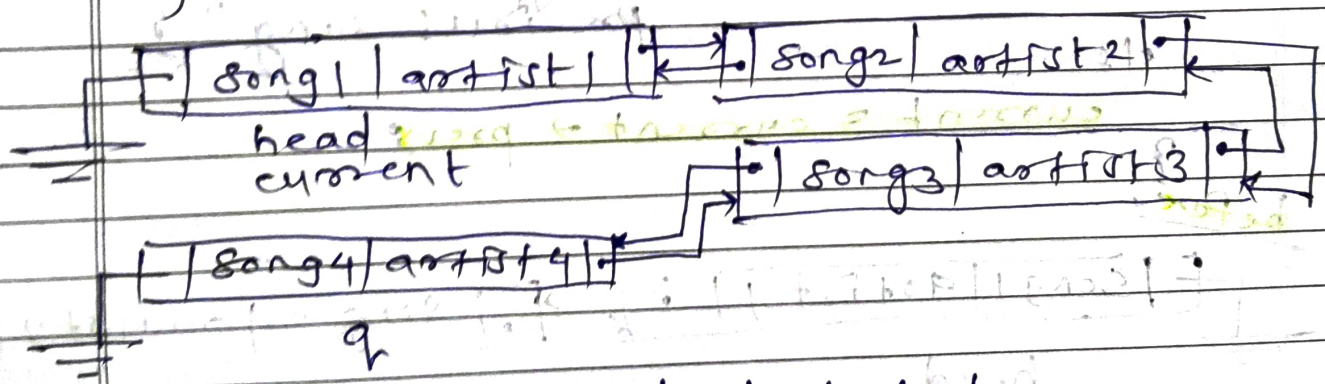


(6) Reverse Traversal : (Show how the playlist can be traversed in the reverse dir" using "Prev" pointer).



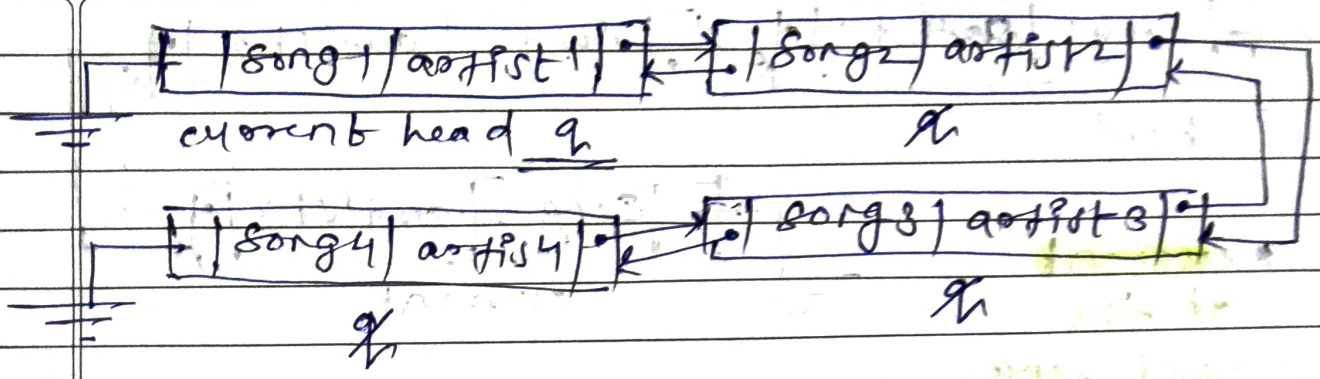
let pointer q = head
while (q → next != NULL)
q = q → next.

By this "q" will be at last node.



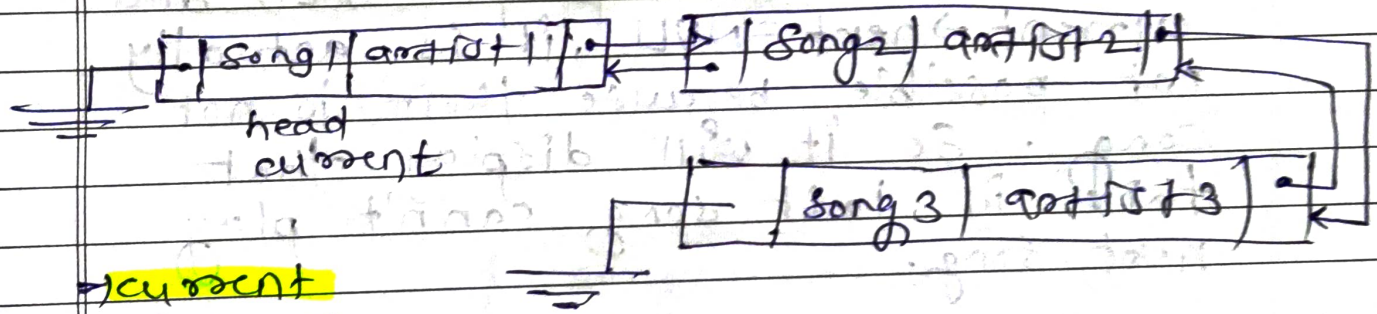
So now q is at last node.
to move it in reverse manner. ↓

while (q → prev != NULL)
q = q → prev;



q moved in reverse direction (reverse traversal) using pointer prev.

* (7) Special cases:- Show appropriate diagrams for at least one boundary condⁿ.



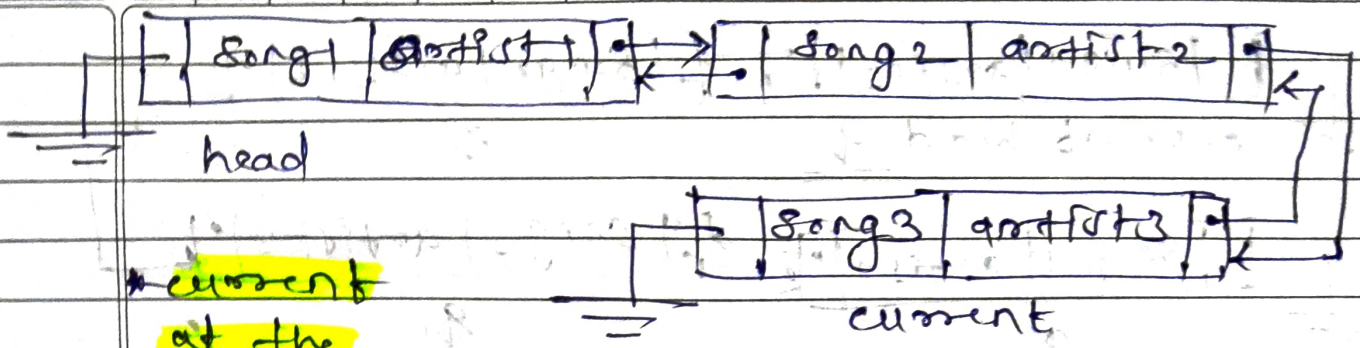
→ current at first song.

① If at this position we try to navigate current to previous song, it is not possible.

② So we will display this is first song can't go to previous song.

③ If we try to navigate current in forward direction (at this position) current will become current of next
i.e current = current → next.

```
void remove(string x)
{
if(empty())                    //delete function
```

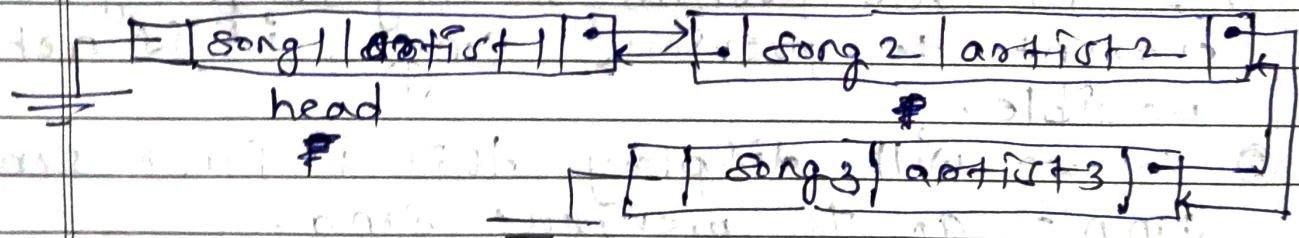


* current at the last song.

① let us consider initially current was at head position and it is traversed through playlist using void play("next") till current is at song 3.

② After that if void play("next") and current->next=NULL, it is actually not possible because their is not song. So it will display that this is last song, can't play next song.

⇔ Deletion of the first song.



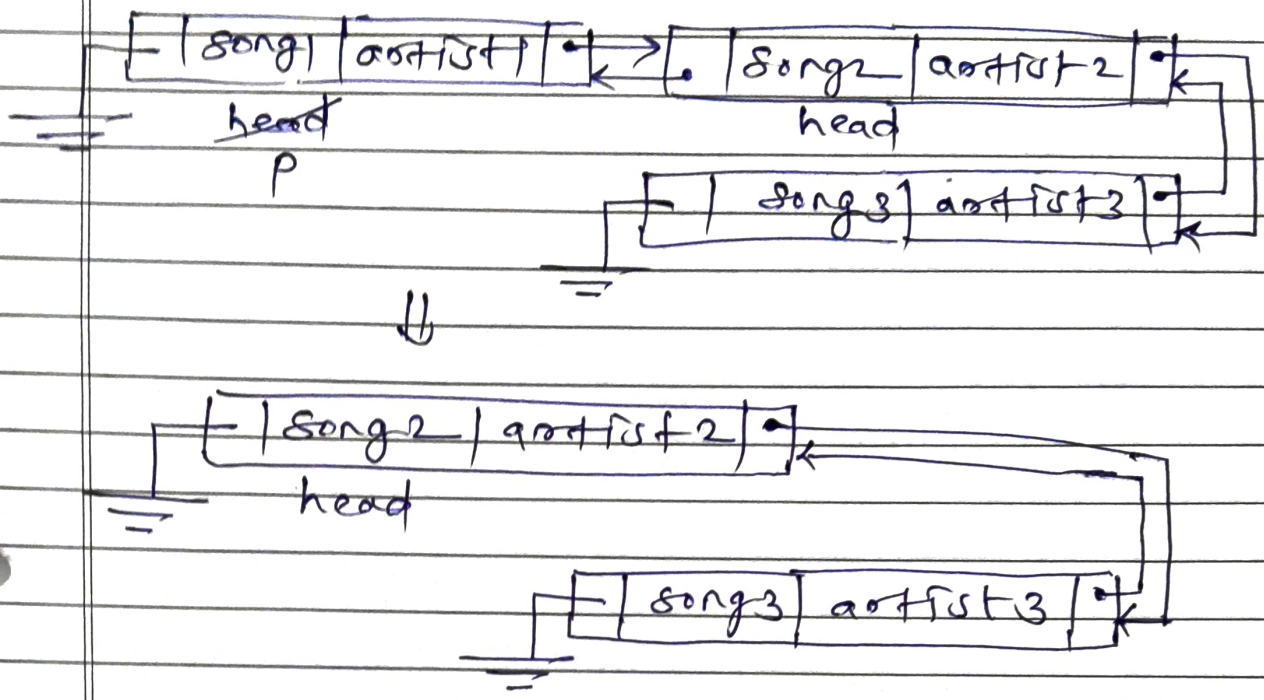
① if head song (node) is to be deleted point p = head ~~->next~~;

② head = head->next;

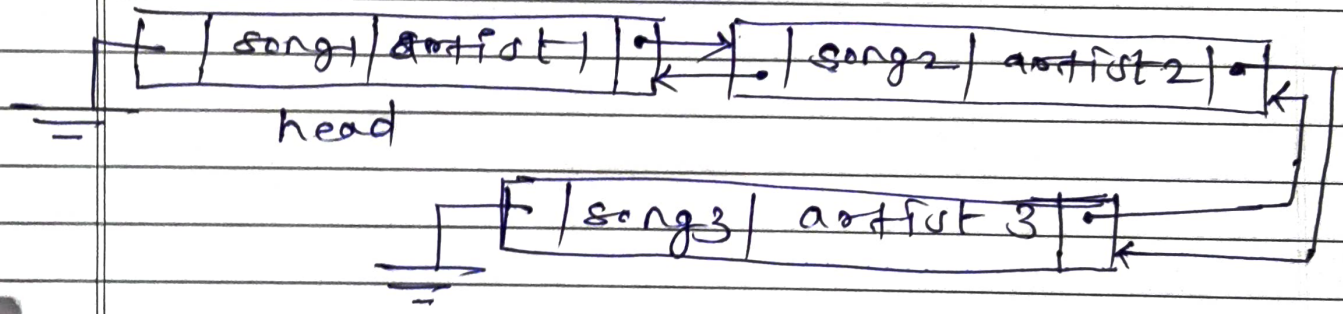
③ head->prev = NULL;

④ delete p;

return;



* deletion of last song :



① to delete last song point q pointer at head.
② q = head;
③ while (q→next != NULL)
q = q→next
④ due to this q will be at last song.
⑤ then point p pointer i.e q→prev = p:
p = q→prev;
p→next = NULL;
delete q;